

Actividad 5. Aprendizaje No Supervisado con Meta AI.

Prompt 1. ¿Cómo funciona matemáticamente el algoritmo K-Means paso a paso? Explica el rol de los centroides, la inercia y el proceso iterativo de asignación y actualización de clusters. ¿Por qué K-Means siempre converge pero no siempre encuentra el óptimo global?

K-Means: Optimización disfrazada de Clustering

K-Means es optimización disfrazada de clustering. Quieres partir (n) puntos $(x_1, \dots, x_n)$ en (K) grupos tal que los puntos dentro de cada grupo estén lo más cerca posible de su centro.

La función que mide eso se llama **inercia** o **WCSS (Within-Cluster Sum of Squares)**:

$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

donde:

- (C_i) es el conjunto de puntos del cluster (i) .
- (μ_i) es el centroide del cluster (i) .
- $(\|x - \mu_i\|^2)$ es la distancia euclidiana al cuadrado.

Minimizar esa suma es el objetivo completo de K-Means.

Rol de los centroides

Un centroide no es simplemente "un punto central". Es la **media aritmética** de los puntos de su cluster:

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

¿Por qué la media?

Porque si fijas un grupo de puntos, el punto μ que minimiza

$$\sum |x - \mu|^2$$

es exactamente la media aritmética de esos puntos.

Esto se obtiene derivando respecto a μ , igualando a cero y resolviendo. Por esta razón, K-Means está estrechamente ligado a la distancia euclidiana.

Algoritmo paso a paso

K-Means consiste en un ciclo de dos operaciones que se alternan.

0. Inicialización

Se eligen K centroides iniciales:

$$\mu_1^{(0)}, \dots, \mu_K^{(0)}$$

Una inicialización aleatoria funciona, pero **k-means++** selecciona puntos alejados entre sí, lo que suele acelerar la convergencia y mejorar los resultados.

1. Asignación

Cada punto se asigna al centroide más cercano:

$$\mu_i(t)$$

$$\left\{ x : |x - \mu_i(t)| \leq |x - \mu_j(t)| \quad \forall j \right\}$$

Esta etapa genera una partición del espacio similar a un **diagrama de Voronoi**.

2. Actualización

Cada centroide se recalcula como la media de los puntos asignados a su cluster:

$$\mu_i(t+1)$$

$$\frac{1}{|C_i(t)|} \sum_{x \in C_i(t)} x$$

3. Repetir

Se repiten los pasos de asignación y actualización hasta que ocurra alguna de las siguientes condiciones:

- Los centroides dejan de moverse.
- Las asignaciones de los puntos ya no cambian.
- La disminución de la inercia es menor que una tolerancia predefinida.

Cada iteración tiene una complejidad computacional de:

$$O(n \cdot K \cdot d)$$

donde d es el número de dimensiones.

¿Por qué siempre converge?

La clave es que la inercia nunca puede aumentar.

Durante la asignación

Un punto cambia de cluster únicamente si el nuevo centroide está más cerca:

$$\begin{aligned} &[\\ &||x - \mu_b||^2 < ||x - \mu_a||^2 \\ &] \end{aligned}$$

Por lo tanto, la suma total de distancias disminuye.

Durante la actualización

Reemplazar un centroide por la media de sus puntos produce el valor que minimiza la suma de distancias cuadráticas dentro del cluster.

Por definición, la inercia disminuye o permanece igual.

Conclusión

- La inercia está acotada inferiormente por (0) .
- Solo existe un número finito de particiones posibles de (n) puntos en (K) grupos.
- La secuencia de valores de inercia es monótonamente decreciente.

Por lo tanto, el algoritmo necesariamente se estabiliza después de un número finito de iteraciones.

¿Por qué no encuentra el óptimo global?

Convergencia no implica optimalidad.

K-Means realiza un descenso codicioso sobre una función objetivo no convexa.

Dependencia de la inicialización

Si dos centroides iniciales caen dentro del mismo cluster real, el algoritmo puede quedar atrapado en una mala partición y nunca alcanzar la mejor solución.

Mínimos locales

La función de inercia contiene numerosos mínimos locales. K-Means converge al mínimo local más cercano a la inicialización elegida.

Problema NP-hard

Encontrar el mínimo global exacto del problema de K-Means es un problema **NP-hard** para conjuntos de datos grandes.

Por esta razón, K-Means debe entenderse como una heurística eficiente, no como un método exacto.

Suposiciones implícitas

K-Means funciona mejor cuando los clusters son:

- Aproximadamente esféricos.
- De tamaño similar.
- Con varianzas parecidas.

Si los datos contienen grupos alargados, con densidades distintas o formas complejas, minimizar la inercia puede no corresponder a un "buen" clustering desde el punto de vista práctico.

Estrategia utilizada en la práctica

Para reducir la probabilidad de quedar atrapado en un mal mínimo local, se ejecuta K-Means múltiples veces (por ejemplo, 10, 50 o 100 ejecuciones) con inicializaciones distintas.

Finalmente se selecciona la solución con la menor inercia:

$$\min_r \text{Inercia}^{\{r\}}$$

$$\left(\text{Inercia}^{\{r\}} \right)$$

\right)

\]

Esto no garantiza alcanzar el óptimo global, pero mejora considerablemente la calidad de la solución obtenida.

Conclusiones: El algoritmo repite dos pasos:

- 1. Asignación: cada punto va al centroide más cercano.**
- 2. Actualización: cada centroide se recalcula como la media de sus puntos.**

Converge siempre porque cada paso solo puede bajar la inercia, y la inercia no puede ser menor que cero. Como hay un número finito de particiones, el proceso se estabiliza.

No garantiza el óptimo global porque depende de dónde empiezan los centroides y porque la función tiene muchos mínimos locales. K-Means es codicioso: baja por el valle más cercano, no explora todo el espacio. Por eso se ejecuta varias veces con inicios distintos y se elige la mejor inercia.

Prompt 2. ¿Cómo puede utilizarse el clustering para detectar anomalías o fraudes en datos financieros? ¿Qué limitaciones tiene K-Means para este tipo de problemas y qué algoritmos alternativos (como DBSCAN o Isolation Forest) se utilizan en la industria para superar esas limitaciones?

El clustering detecta fraude porque no busca la etiqueta "fraude", busca lo que se parece. En datos financieros tomas cada transacción como un vector — monto, hora, categoría de comercio, dispositivo, IP, velocidad entre compras, historial del usuario — y dejas que el algoritmo agrupe lo que se repite.

Lo normal forma nubes densas: la compra del súper los viernes en la tarde, la nómina el día 15, los retiros en el mismo cajero. Lo raro queda suelto, lejos del

centro de cualquier nube o en una micro-nube de 2-3 puntos. Esa es la idea que usa la banca: la detección de anomalías "identifica valores atípicos en datos mediante análisis de clústeres, permitiendo detectar fraudes o patrones nuevos" sin necesidad de datos etiquetados.

Dónde K-Means se atora

K-Means sigue siendo el primer intento en muchos cursos, pero en fraude tiene cinco fricciones reales:

- **Tienes que decirle cuántos clusters hay (k).** En una cartera real no sabes si hay 5, 50 o 500 comportamientos normales, y cada cliente nuevo cambia la cuenta.
- **Asume bolas perfectas del mismo tamaño.** Obliga a que todos los clusters sean esféricos y con varianza similar. Una compra fraudulenta de madrugada por \$12,000 no es una bola, es un punto aislado.
- **Los outliers lo rompen.** Como el centroide se recalcula con la media, un fraude grande jala el centro y se "camufla". En un estudio comparativo reciente, "K-Means clustering fails to separate fraud cases" mientras modelos supervisados sí lo hacían.
- **No entiende desbalance.** El fraude suele ser <0.5% del volumen. K-Means reparte todo en k grupos, así que el fraude termina absorbido en un cluster normal. Por eso surgió AD-KMeans, que "ajusta dinámicamente el número de clusters según varianza y densidad" y logra 91.2% de recall frente al K-means tradicional.
- **Todo punto pertenece a algo.** No hay concepto de "ruido", así que nunca te dice "esto no encaja en nada". Variantes como BPW K-means mejoran 38% sobre el clásico justo porque ponderan features, pero siguen arrastrando la limitación base.

En la práctica, K-Means sirve para una primera segmentación de clientes, no para cazar el 0.3% raro.

Lo que sí se usa en industria

1. DBSCAN y HDBSCAN — densidad, no forma

No pides k , pides "qué tan cerca" (eps) y "cuántos vecinos mínimos". Todo lo que no alcanza densidad queda marcado como ruido, y ese ruido es tu alerta.

- Detecta clusters con formas arbitrarias: anillos de compras en gasolineras, cadenas de transferencias rápidas.
- Un framework híbrido reciente usa "DBSCAN-based data augmentation" y reporta 99.5% recall y 99.8% F1 en tarjetas, superando ensembles tradicionales.
- Es el favorito para grafos de comercios o geolocalización porque aguanta densidades variables mejor que K-Means.

Limitación: elegir eps en datos de 50 dimensiones duele, por eso en producción se pasa a HDBSCAN, que construye la jerarquía y corta solo.

2. Isolation Forest — no es clustering, pero es el estándar

En vez de agrupar, aísla. Construye árboles aleatorios y mide cuántas particiones necesitas para separar un punto. Un fraude se aísla en 3-4 cortes; una compra normal necesita 20.

- Muy rápido, escala a millones de transacciones, no asume distribución.
- Arquitecturas de banca digital en tiempo real con Kafka + Spark lo usan y "achieves 99% accuracy in two datasets, outperforming traditional machine learning methods".
- En comparativas directas, Isolation Forest alcanzó 99.72% de accuracy frente a 99.62% de LOF en tarjetas.
- Los bancos lo están volviendo explicable: una mejora reciente lo integra con árboles de decisión para generar reglas 'si-entonces' y dar transparencia al auditor.

Consejo operativo que repiten los equipos: entrena solo con normales, fija contamination entre 0.01 y 0.05, y ajusta el umbral según tolerancia a falsos positivos.

3. LOF (Local Outlier Factor)

Mide densidad local contra vecinos. Útil cuando el fraude imita el monto pero rompe el ritmo, por ejemplo 5 micro-compras en 2 minutos desde un dispositivo nuevo. No escala tan bien como Isolation Forest, pero complementa.

4. Híbridos que ves en fintech real

- Proyectos open de fintech combinan "Isolation Forest and DBSCAN models to identify fraudulent transactions" y añaden SHAP para explicar por qué saltó la alerta.
- En compras empresariales se usa "a hybrid approach combining clustering and Isolation Forest for anomaly detection", evaluando con Silhouette y priorizando casos para auditoría.

Cómo elegir

Situación	Mejor opción	Por qué
Necesitas marcar "esto no pertenece a nada" sin etiquetas	DBSCAN / HDBSCAN	Etiqueta ruido directamente
Millones de transacciones en streaming, pocos fraudes	Isolation Forest	$O(n \log n)$, maneja desbalance
Fraude que imita monto pero cambia vecindario	LOF	Compara densidad local
Solo tienes K-Means disponible	Usa variante adaptativa (AD-KMeans, BPW) y combínalo con Isolation Forest	Reduce arrastre de centroides

En México, donde muchos fraudes son por toma de cuenta y compras nocturnas en e-commerce, la receta más común es: primero HDBSCAN para segmentar comportamiento por usuario, luego Isolation Forest por segmento con features de velocidad, dispositivo y geodistancia, y finalmente un revisor humano guiado por las reglas explicables.

Conclusiones: El clustering detecta fraude agrupando lo que se repite: compras con mismo horario, monto, comercio o dispositivo forman nubes densas de comportamiento normal, y lo que queda fuera o aislado se marca como anomalía, sin necesidad de etiquetas previas. K-Means tiene limitaciones para esto porque obliga a definir el número de clusters, asume formas

esféricas del mismo tamaño, es sensible a los outliers que mueven los centroides, no tiene concepto de ruido y en datos muy desbalanceados, donde el fraude es menos del 1 por ciento, termina absorbiendo los casos raros dentro de un cluster normal.

Prompt 3. ¿Cuáles son los 3 casos de uso más impactantes del aprendizaje no supervisado en la industria actual? Para cada uno, explica qué tipo de algoritmo se utiliza (clustering, reducción de dimensionalidad, detección de anomalías) y qué valor de negocio genera para las empresas.

Los tres usos que hoy mueven más dinero con aprendizaje no supervisado no necesitan etiquetas: encuentran lo raro, agrupan lo parecido y comprimen lo enorme. Cada uno se apoya en una familia distinta de algoritmos.

1. Detección de anomalías — fraude, ciberseguridad y riesgo operativo

Qué algoritmo se usa

- **Isolation Forest:** crea árboles de decisión aleatorios y aísla los puntos; los que se separan en pocas particiones reciben un score de anomalía entre 0 y 1, y por encima de 0,5 se marcan como raros.
- **One-Class SVM:** aprende una frontera alrededor de lo “normal” con datos sin etiquetar; todo lo que cae fuera se etiqueta como anomalía.
- **Autoencoders y K-means:** redes neuronales que reconstruyen el comportamiento habitual, y K-means que agrupa lo normal para señalar lo que queda lejos del centroide.

Dónde impacta

- **Banca y pagos:** los modelos entrenados con datos globales aumentan la detección de fraude de 2 a 3 veces y suben las aprobaciones hasta 7,4%

sobre las defensas actuales. Nilson proyecta pérdidas por fraude con tarjeta de \$408,5 mil millones entre 2020 y 2030, así que cada punto de mejora evita miles de millones.

- **Ciberseguridad:** sistemas de detección de intrusiones monitorean tráfico de red en tiempo real y marcan picos o patrones inesperados como posibles violaciones.
- **Manufactura:** sensores no etiquetados alimentan modelos que predicen fallas antes de la avería crítica, reduciendo tiempo de inactividad.

Valor de negocio: menos pérdidas directas, menos revisiones manuales (los analistas se enfocan solo en casos de alto riesgo), y protección continua sin necesidad de re-etiquetar fraudes nuevos.

2. Clustering — segmentación de clientes y personalización

Qué algoritmo se usa

- **K-means:** el método más usado; separa puntos en K grupos alrededor de centroides, recalculando hasta estabilizar. Es la técnica de clustering “con diferencia, la más utilizada” para agrupar automáticamente puntos con características similares.
- **Clustering jerárquico (aglomerativo) y DBSCAN:** útiles cuando la jerarquía importa o cuando los grupos tienen formas irregulares; funcionan bien en conjuntos medianos y permiten interpretar relaciones entre segmentos.
- K-means también se aplica directamente a segmentación de clientes y detección de anomalías según IBM.

Dónde impacta

- **Retail y e-commerce:** dividir por ingresos, frecuencia y gasto (modelo RFM) permite campañas hiper-personalizadas. Casos como AJIO lograron un aumento de 4X en conversiones mediante segmentación basada en comportamiento.

- **Streaming, telecom y banca:** recomendaciones de contenido y ofertas se basan en clusters, no en reglas fijas.

Valor de negocio: aumenta la tasa de conversión y el ticket promedio, reduce el gasto en audiencias frías, y mejora retención porque el mensaje encaja con el perfil real, no con suposiciones demográficas amplias.

3. Reducción de dimensionalidad — compresión, visualización y preprocesamiento industrial

Qué algoritmo se usa

- **PCA (Análisis de Componentes Principales):** transforma variables correlacionadas en componentes principales que conservan la mayor parte de la información original. Calcula vectores propios de la matriz de covarianza para proyectar datos a menos dimensiones.
- **t-SNE, UMAP y PCA de kernel:** variantes no lineales para visualizar clústeres en 2D/3D cuando PCA lineal no basta.

Dónde impacta

- **Preprocesamiento de modelos:** PCA extrae las características más informativas y reduce la “maldición de la dimensionalidad”, donde cada nueva variable degrada el rendimiento.
- **Control de calidad y energía:** al proyectar datos de alta dimensión, PCA minimiza multicolinealidad y ajuste excesivo, problemas que hacen que los modelos fallen con datos nuevos. En parques eólicos, por ejemplo, se usan SVD y correlación para reducir a 8 patrones espaciales y predecir potencia total.
- **Imágenes y salud:** PCA crea representaciones compactas para almacenamiento y transmisión, y ayuda a visualizar datos complejos en 2D/3D.

Valor de negocio:

- reduce costos de cómputo y almacenamiento (menos variables = modelos más rápidos),
- mejora precisión al eliminar ruido y redundancia,
- permite a equipos no técnicos “ver” patrones en tableros, acelerando decisiones en mantenimiento, marketing y riesgo.

Conclusiones: La detección de anomalías protege ingresos, clustering los multiplica al hablarle a cada cliente como único, y la reducción de dimensionalidad hace que todo lo anterior sea posible a escala industrial sin ahogarse en datos. Los tres comparten la misma ventaja del no supervisado: aprenden de lo que ya tienes, sin esperar meses de etiquetado.